Ananya Yallapragada
019142383

Homework 9 Report

Section 0 - Personal Configuration and Domain

Value	Definition
SID4	2383
PORT_BASE	8583
PREFIX	s2383
SEED	2383
VERIFY_SEED	262383
DOMAIN_ID	7
Hardware	MacBook Pro, Apple M4 (10-core CPU), 16 GB RAM
Python version	3.11.9
Tagged commit hash	75ecfaf239f9c805e1c2cbd9f6cb3c05765598da

Github - https://github.com/Lyra0710/data260-2383/tree/hw9
(Please note that since the commit hash needs to be included in the report, a new commit will be created when the report is added again with the hash included. The tag link here is before the hash was added. The updated tag is pasted in the comments on canvas.).

Ananya Yallapragada
019142383

Part 1

fixture_view_cache.py

```python
logger = logging.getLogger(__name__)

PREFIX = os.getenv("PREFIX", "s2383")


class FixtureViewCache:
    def __init__(self, redis_client: redis.Redis):
        self.redis_client = redis_client

    def increment_fixture_views(self, fixture: "Fixtu
        count = self.redis_client.incr(f"{PREFIX}:vie

        if count % 10 == 0:
            fixture.view_count = count
            db.commit()
            logger.info(
                "Synced views to database: fixture_id=
                fixture.id,
                count,
            )

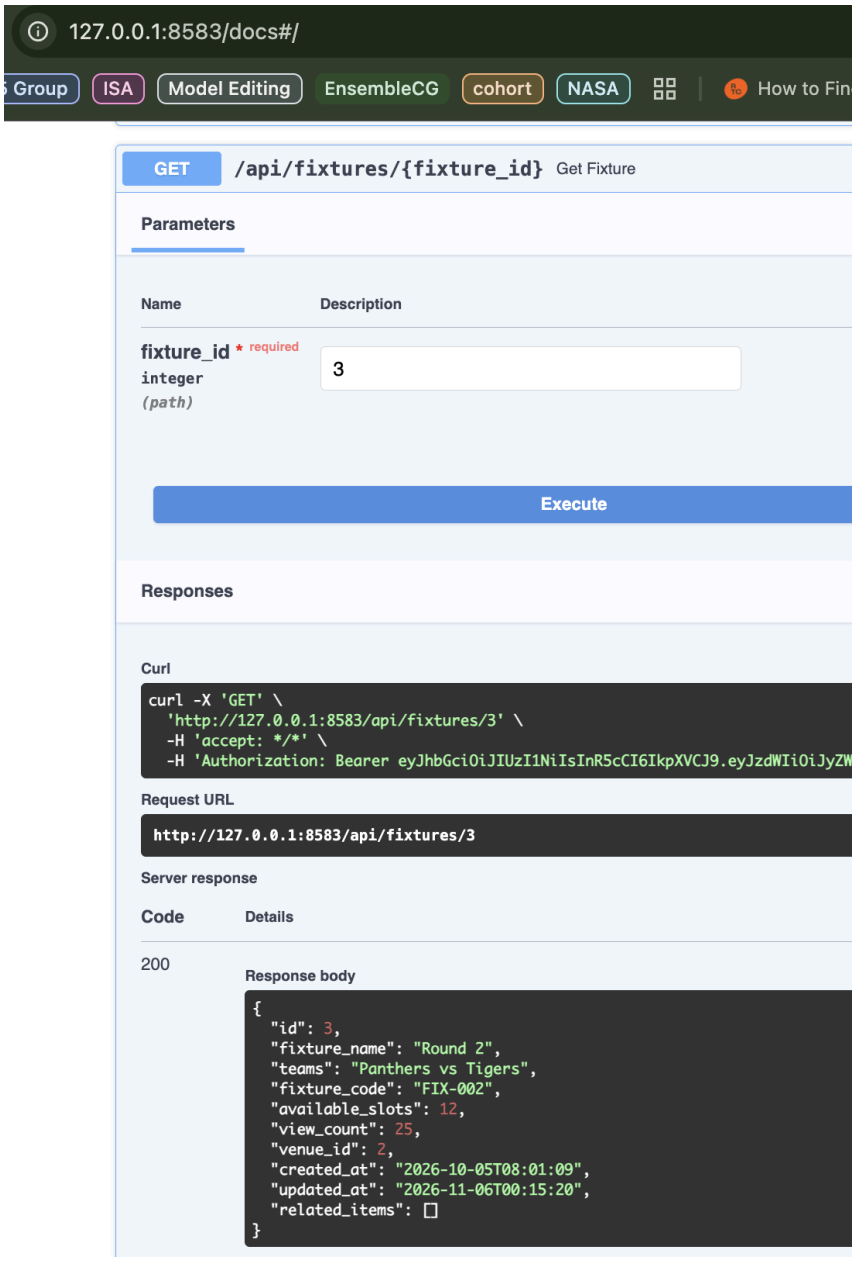
        return count


fixture_view_cache = FixtureViewCache(
    redis.Redis(
        host=os.getenv("REDIS_HOST", "127.0.0.1"),
        port=int(os.getenv("REDIS_PORT", "6379")),
        decode_responses=True,
    )
)
```

main.py

```python
view_count = fixture_view_cache.increment_fixture_views(fixture, db)
return fixture_to_dict(fixture, view_count=view_count)
```

Ananya Yallapragada
019142383



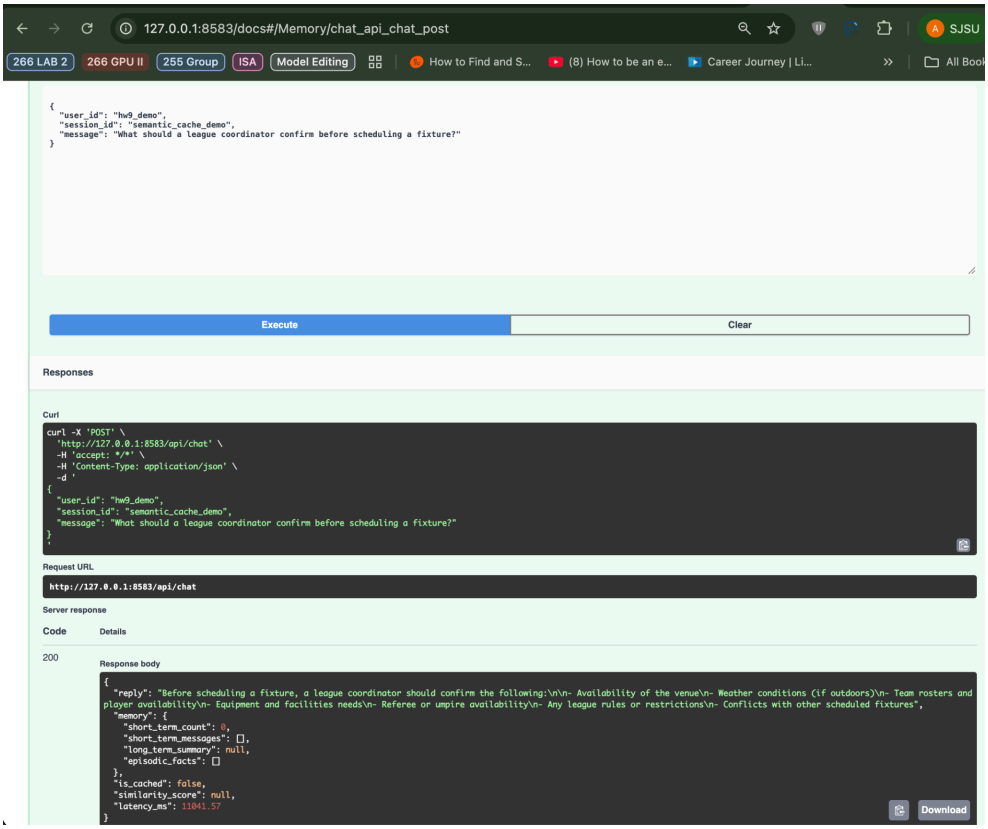

```
(.venv) annie@Siris-MacBook-Pro data260-2383 % docker compose exec redi
s redis-cli GET s2383:views:fixture:3
"25"
```

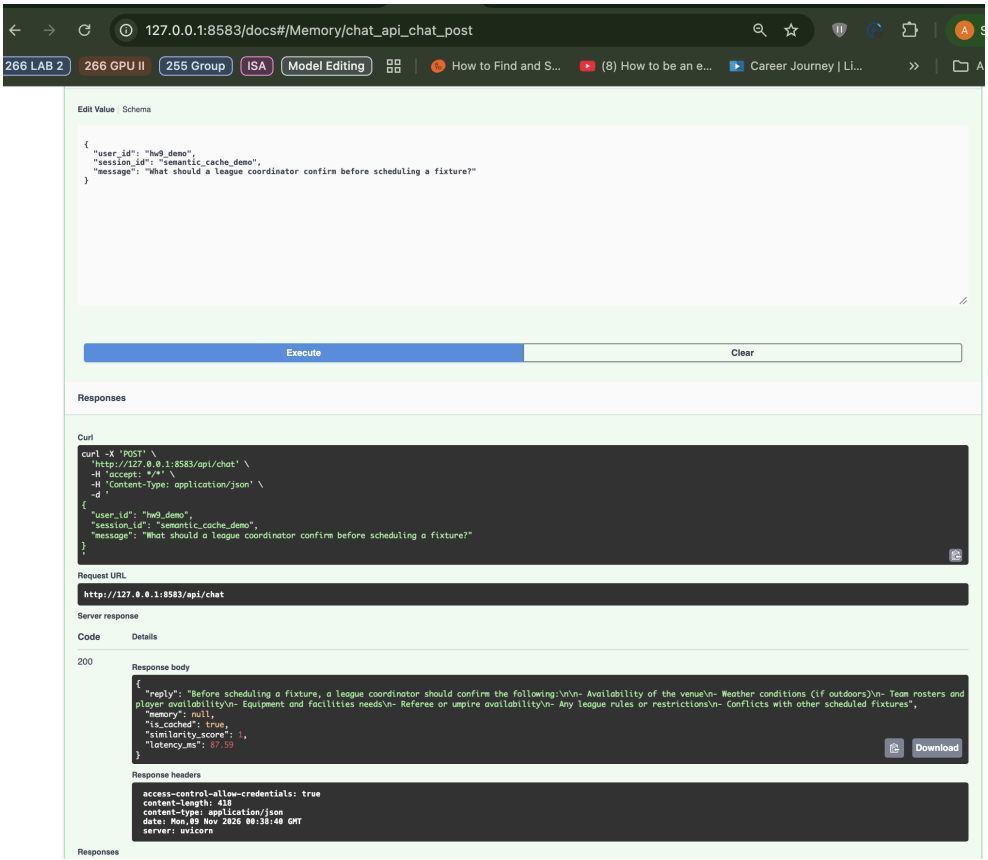
```
(.venv) annie@Siris-MacBook-Pro data260-2383 % /usr/local/mysql-9.4.0-m
acos15-arm64/bin/mysql -u root -p -S /tmp/mysql.sock s2383_rel -e "SELE
CT id, view_count FROM fixtures WHERE id = 3;"
Enter password:
+----+------------+
| id | view_count |
+----+------------+
|  3 |         20 |
+----+------------+
```

```
INFO: Synced views to database: fixture_id=3 view_count=30
INFO:     127.0.0.1:52313 - "GET /api/fixtures/3 HTTP/1.1" 200 OK
```

Ananya Yallapragada
019142383

Part 2: Semantic Response Cache





Ananya Yallapragada
019142383

```
(base) annie@mac web_application % python semantic_cache_demo.py
Query 1: cached=False similarity=None latency_ms=7922.31
Query 2: cached=True similarity=1.0 latency_ms=34.53
Query 3: cached=False similarity=None latency_ms=4265.15
Query 4: cached=False similarity=None latency_ms=9005.79
Query 5: cached=True similarity=1.0 latency_ms=22.41
Query 6: cached=False similarity=None latency_ms=5146.38
Query 7: cached=False similarity=None latency_ms=5773.75
Query 8: cached=False similarity=None latency_ms=13538.18
Query 9: cached=True similarity=1.0 latency_ms=27.28
Query 10: cached=False similarity=None latency_ms=6458.33
{
  "query_count": 10,
  "cache_hits": 3,
  "cache_hit_rate_percent": 30.0,
  "average_cached_latency_ms": 28.07,
  "average_non_cached_latency_ms": 7444.27
}
```

routers/memory.py - ingress hit/miss

```python
@router.post("/chat", response_model=ChatResponse)
def chat(request: ChatRequest):
    started_at = time.perf_counter()
    session_id = request.session_id or request.user_id
    query_embedding = embed_texts([request.message])[0]
    data_version = semantic_response_cache.current_data_version()
    cached_response = semantic_response_cache.find_similar(
        query_embedding,
        data_version,
        request.user_id,
        session_id,
        CACHE_THRESHOLD,
    )

    if cached_response is not None:
        latency_ms = round((time.perf_counter() - started_at) * 1000, 2)
        logger.info(
            "Semantic cache hit: similarity_score=%.4f latency_ms=%.2f",
            cached_response["similarity_score"],
            latency_ms,
        )
        return {
            "reply": cached_response["answer"],
            "memory": None,
            "is_cached": True,
            "similarity_score": cached_response["similarity_score"],
            "latency_ms": latency_ms,
        }

    response = generate_reply(
        request.user_id,
        session_id,
        request.message,
        experiment_id=request.experiment_id,
        experiment_phase=request.experiment_phase,
    )
    semantic_response_cache.store(
        request.message,
        query_embedding,
        response["reply"],
        data_version,
        request.user_id,
        session_id,
    )
    latency_ms = round((time.perf_counter() - started_at) * 1000, 2)
    logger.info("Semantic cache miss: latency_ms=%.2f", latency_ms)
```

Ananya Yallapragada
019142383

```
        return {
            **response,
            "is_cached": False,
            "similarity_score": None,
            "latency_ms": latency_ms,
        }
```

code/web_application/semantic_cache.py - similarity decision

```
    self.ensure_index(len(embedding))
    user_tag = self.escape_tag(user_id)
    session_tag = self.escape_tag(session_id)
    query = (
        Query(
            f"(@user_id:{{{user_tag}}} @session_id:{{{session_tag}}})"
            "=>[KNN 1 @embedding $embedding AS distance]"
        )
        .return_fields("answer", "data_version", "distance")
        .sort_by("distance")
        .dialect(2)
    )
    result = self.redis_client.ft(INDEX_NAME).search(
        query,
        query_params={"embedding": self.vector_bytes(embedding)},
    )
    if not result.docs:
        return None

    match = result.docs[0]
    similarity_score = 1 - float(match.distance)
    if similarity_score < threshold or match.data_version != data_version:
        return None

    return {
        "answer": match.answer,
        "similarity_score": similarity_score,
    }
```

```
            self.ensure_index(len(embedding))
            key = f"{ENTRY_PREFIX}{uuid.uuid4()}"
            self.redis_client.hset(
                key,
                mapping={
                    "query": query,
                    "embedding": self.vector_bytes(embedding),
                    "answer": answer,
                    "data_version": data_version,
                    "user_id": user_id,
                    "session_id": session_id,
                },
            )
```

Ananya Yallapragada
019142383

Part 3: Threshold Economics and Stale Answers

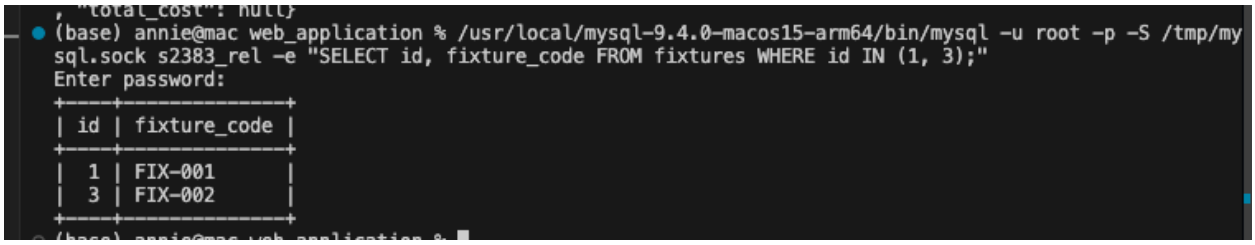
3.2 - Threshold Sweep

Threshold	Hit rate	False-hit rate	Mean latency	Total cost
0.75	75.0%	33.33%	639.40 ms	110
0.80	75.0%	33.33%	792.08 ms	110
0.85	75.0%	33.33%	699.82 ms	110
0.90	55.0%	22.73%	958.40 ms	68
0.95	25.0%	0.0%	1405.19 ms	30

3.3 - One false hit in full

- Cached query: 'What is the fixture code for fixture 1?'
- Incoming query: 'What is the fixture code for fixture 3?'
- Similarity score: 0.876697003841
- Cached answer returned: 'The fixture code for fixture 1 is not provided in the available information.'
- Correct incoming answer: FIX-002
- Why it was a false hit: the embeddings treated the shared wording - 'fixture code' and 'fixture' - as highly similar and did not distinguish fixture IDs 1 and 3 strongly enough.

The cached answer was already incomplete, because the current chat pipeline does not retrieve fixture records before answering.



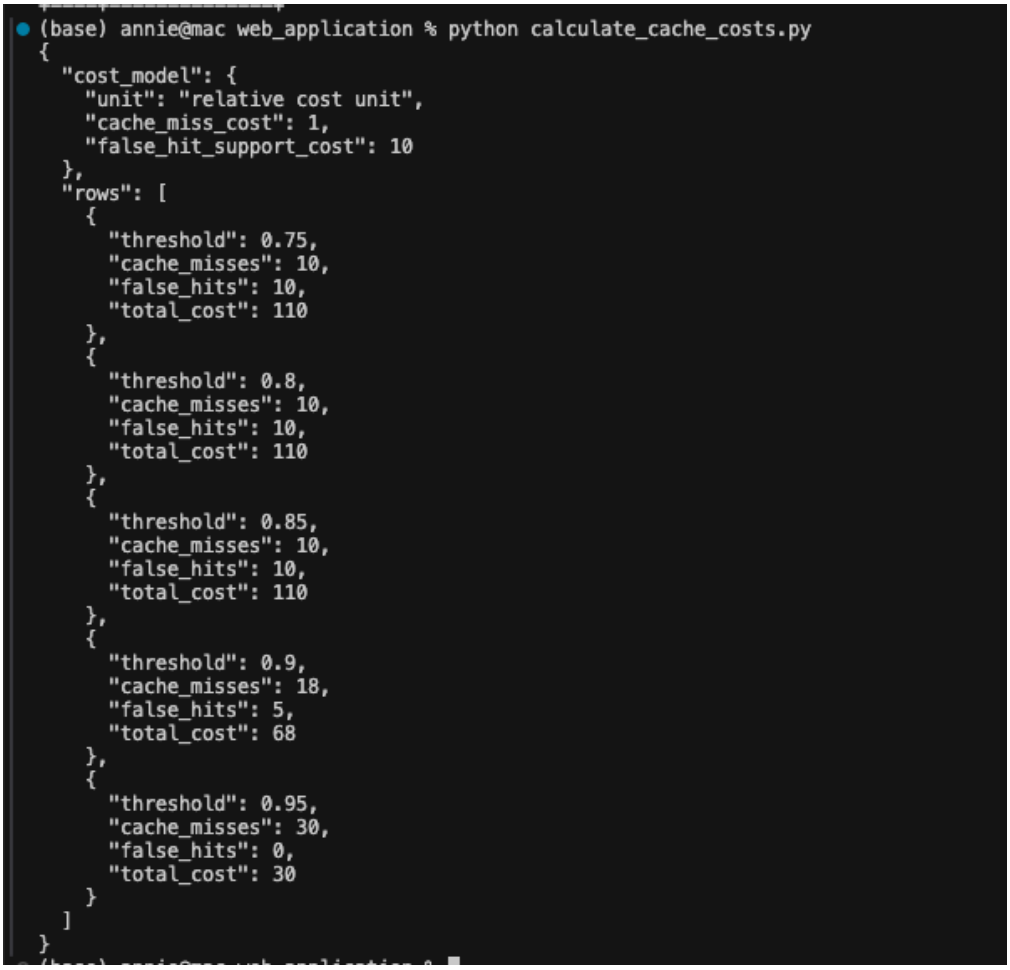
3.4 - Cost model and threshold choice

- Cache miss: 1 relative cost unit - it requires a normal agent/model invocation.
- False hit: 10 relative cost units - it returns an incorrect domain answer and requires correction or support.

Ananya Yallapragada
019142383

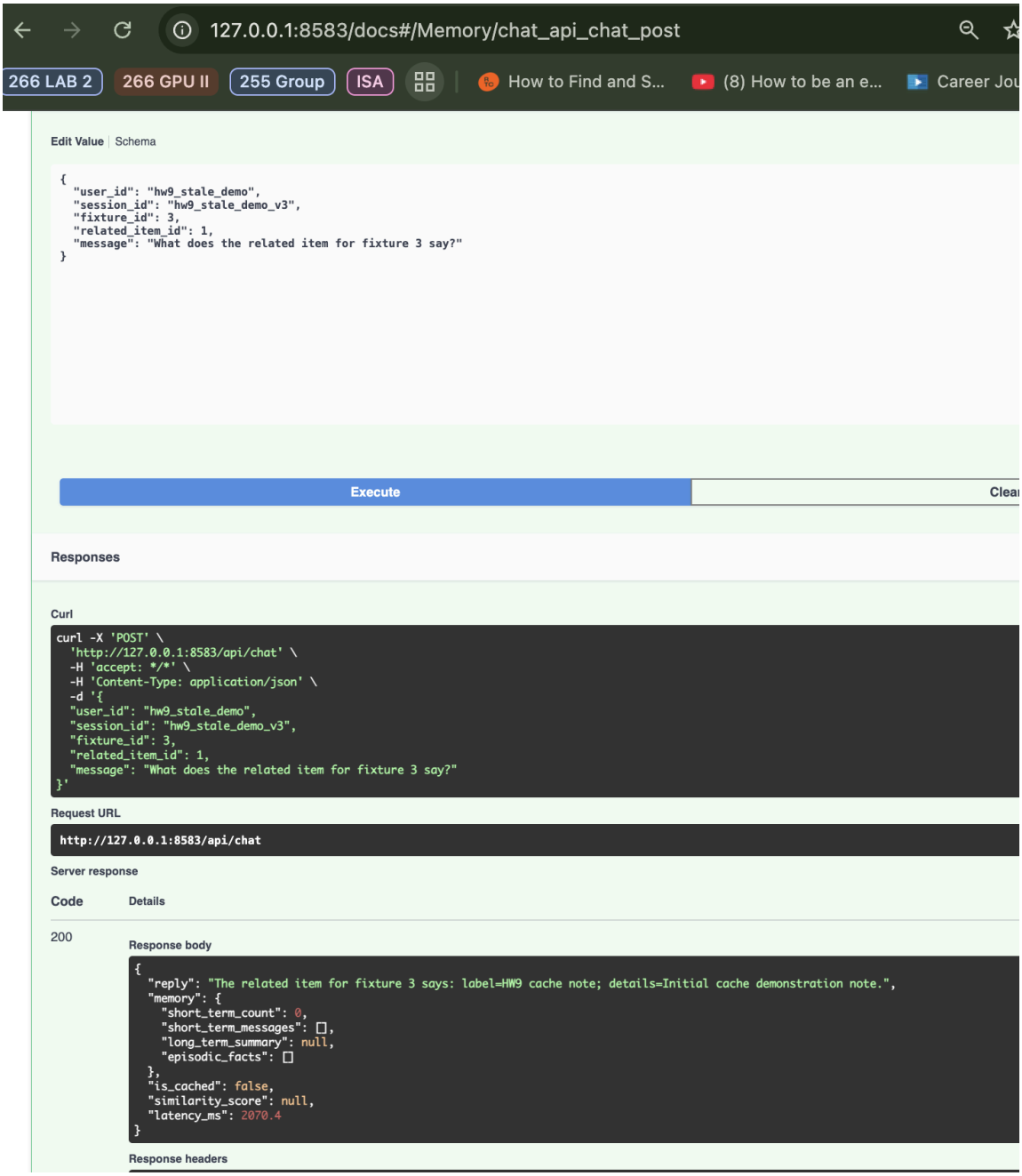
Choosing 0.95 to ship:

- Lowest total cost: **30**
- False-hit rate: **0%**
- Trade-off: it serves only exact duplicates from cache, so its hit rate is only **25%** and mean latency is higher.



Ananya Yallapragada
019142383

3.5 Stale answers and invalidation



Ananya Yallapragada
019142383

Request body required

Edit Value | Schema

```
{
  "user_id": "hw9_stale_demo",
  "session_id": "hw9_stale_demo_v3",
  "fixture_id": 3,
  "related_item_id": 1,
  "message": "What does the related item for fixture 3 say?"
}
```

Execute | Clear

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8583/api/chat' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
  "user_id": "hw9_stale_demo",
  "session_id": "hw9_stale_demo_v3",
  "fixture_id": 3,
  "related_item_id": 1,
  "message": "What does the related item for fixture 3 say?"
}'
```

Request URL

```
http://127.0.0.1:8583/api/chat
```

Server response

Code	Details
200	Response body

```
{
  "reply": "The related item for fixture 3 says: label=HW9 cache note; details=Initial cache demonstration note.",
  "memory": null,
  "is_cached": true,
  "similarity_score": 0.9999998807907104,
  "latency_ms": 379.89
}
```

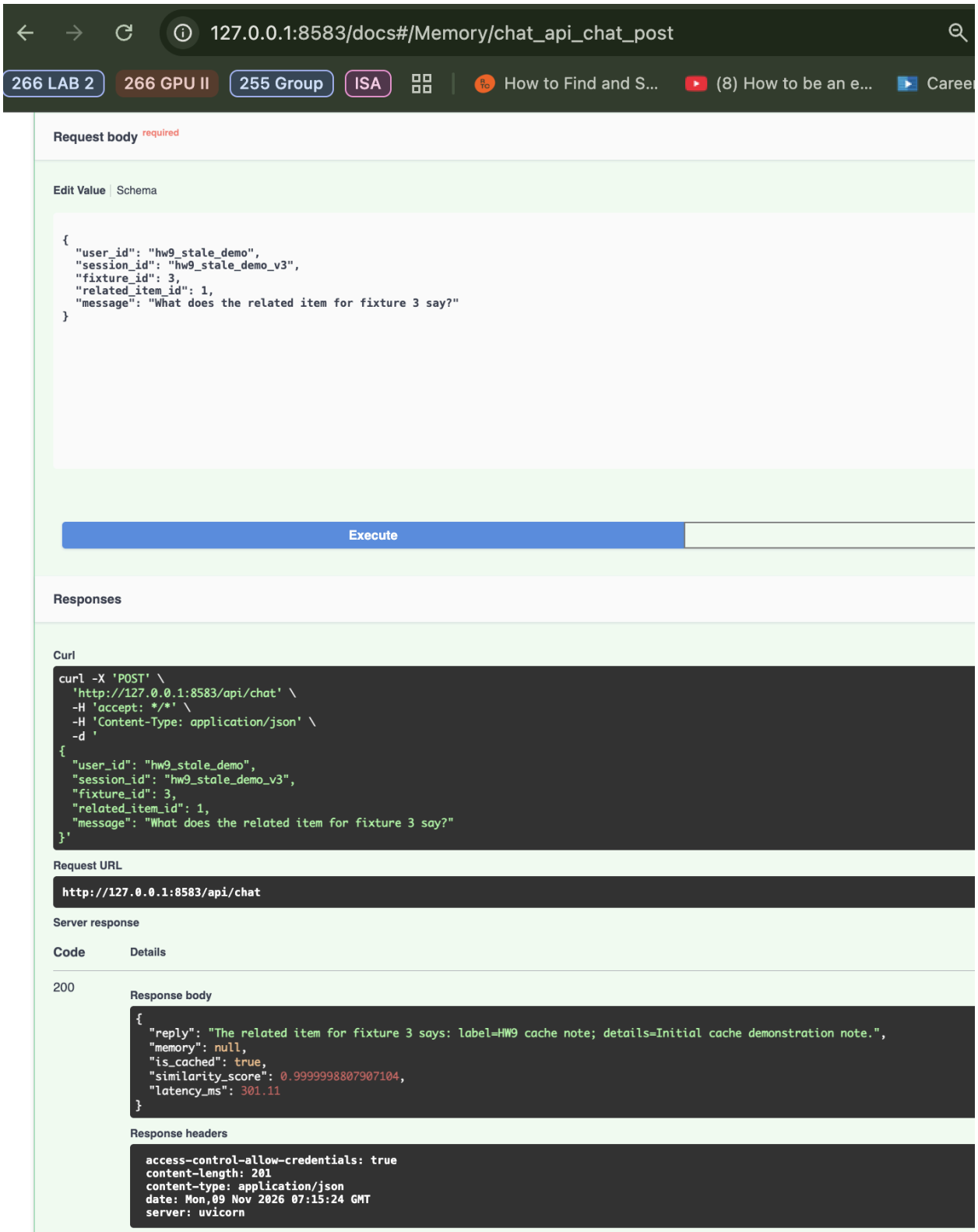
Response headers

```
access-control-allow-credentials: true
content-length: 201
content-type: application/json
date: Mon,09 Nov 2026 07:10:01 GMT
server: uvicorn
```

Responses

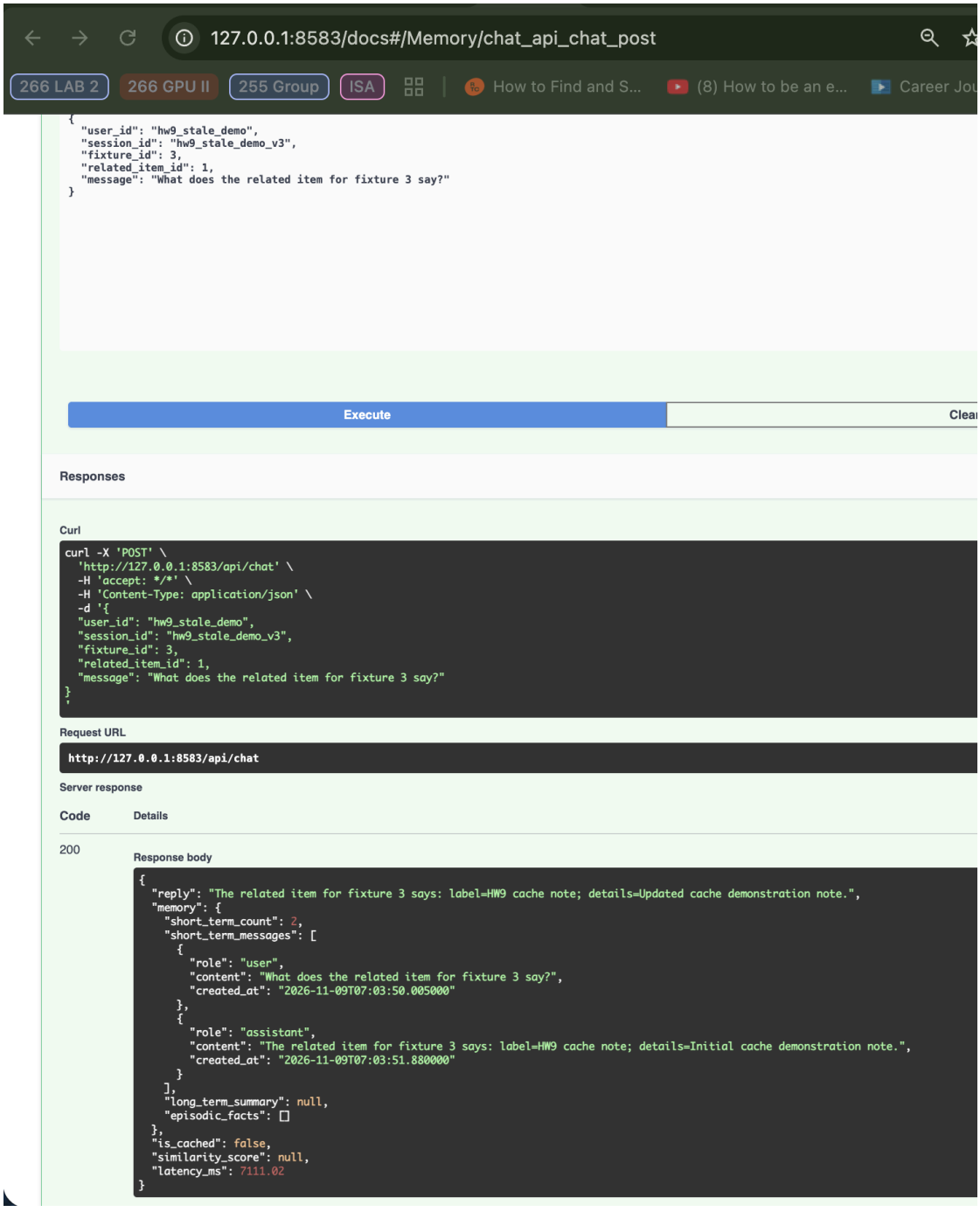
Ananya Yallapragada
019142383

Making the database record stale



Ananya Yallapragada
019142383

Invalidate cached entries by advancing the stored data-version marker



The stale cache hit returned 'Initial cache demonstration note' after the database record had changed. Incrementing s2383:semantic-cache:data-version caused the next request to miss the cache and return 'Updated cache demonstration note.' The strategy is simple and prevents stale

answers, but it invalidates all semantic-cache entries logically, so unrelated cached answers also become misses after any record change.

semantic_cache.py - the stored data-version marker and INCR invalidation.

```python
def current_data_version(self) -> str:
    self.redis_client.setnx(DATA_VERSION_KEY, "0")
    return str(self.redis_client.get(DATA_VERSION_KEY))

def bump_data_version(self) -> str:
    return str(self.redis_client.incr(DATA_VERSION_KEY))
```

the cache rejecting an entry whose stored data version no longer matches

```python
result = self.redis_client.ft(INDEX_NAME).search(
    query,
    query_params={"embedding": self.vector_bytes(embedding)},
)
if not result.docs:
    return None
```

Part 4 - Self-Improving Multi-Agent System

```
(base) annie@mac web_application % python multi_agent_system.py
{
  "Baseline": {
    "correct_answers": 0,
    "incorrect_answers": 3,
    "average_attempts": 1.0,
    "average_latency_ms": 0.0
  },
  "Evaluator Repair": {
    "correct_answers": 3,
    "incorrect_answers": 0,
    "average_attempts": 2.0,
    "average_latency_ms": 0.0
  },
  "Lesson Memory": {
    "correct_answers": 3,
    "incorrect_answers": 0,
    "average_attempts": 1.0,
    "average_latency_ms": 0.0
  },
  "Validated Improvement": {
    "correct_answers": 3,
    "incorrect_answers": 0,
    "average_attempts": 1.0,
    "average_latency_ms": 0.0
  }
}
```

Ananya Yallapragada
019142383

Multi_agent_system.py -

```python
    if use_evaluator and initial_evaluation["result"] == "REVISE":
        if save_lessons:
            create_lesson(initial_evaluation, lessons)
        final_answer = answering_agent.answer(
            record,
            facts,
            active_lessons,
            feedback=initial_evaluation["feedback"],
            attempt=2,
        )
        final_evaluation = evaluator_agent.evaluate(
            record["question"],
            final_answer,
            facts,
        )
        attempts = 2
```

```python
def validation_passes(before: list[dict], after: list[dict]) -> bool:
    for previous, current in zip(before, after):
        if previous["final_evaluation"]["result"] == "PASS" and (
            current["final_evaluation"]["result"] != "PASS"
        ):
            return False
    return summarize(after)["correct_answers"] >= summarize(before)["correct


def main() -> None:
    facts = load_json(REFERENCE_PATH)["facts"]
    training = load_json(TRAINING_PATH)
    validation = load_json(VALIDATION_PATH)
    test = load_json(TEST_PATH)

    lessons = []
    training_results = run_dataset(training, facts, lessons, True, False, Tr
    validation_before = run_dataset(validation, facts, [], False, False, Fal
    validation_after = run_dataset(validation, facts, lessons, False, True,
    keep_lessons = validation_passes(validation_before, validation_after)
    selected_lessons = lessons if keep_lessons else []
    LESSONS_PATH.write_text(
        json.dumps(selected_lessons, indent=2) + "\n",
        encoding="utf-8",
    )

    variants = {
        "Baseline": (False, False),
        "Evaluator Repair": (True, False),
        "Lesson Memory": (False, True),
        "Validated Improvement": (False, True),
    }
```

Ananya Yallapragada
019142383

The Answering Agent generates an answer from the input question. The Evaluator Agent checks it against reference_data.json and returns PASS or REVISE with feedback. When revision is required, the Answering Agent gets one additional attempt. Lessons are saved in lessons.json and used before later answers. Lessons are retained only when validation does not regress previously correct results.

One complete revision sequence:

Step	**Result**
Question	What is the fixture code for fixture 1?
Initial answer	I do not have a verified value for fixture 1.
Evaluator result	REVISE
Feedback	The fixture 1 code is FIX-001.
Revised answer	The fixture 1 code is FIX-001.
Final evaluation	PASS
Attempts	2

Lesson reuse and validation:

The system saved lessons for omitted fixture_code and detailed values. The fixture_code lesson was reused twice. Validation improved from 1/4 correct before lessons to 4/4 correct after lessons. The unsupported venue question remained a correct ‘no verified answer’ response, so the lessons were retained.

Variant	**Correct**	**Incorrect**	**Average attempts**	**Average latency**
Baseline	0	3	1.0	0.0 ms
Evaluator Repair	3	0	2.0	0.0 ms

Ananya Yallapragada
019142383

Variant	Correct	Incorrect	Average attempts	Average latency
Lesson Memory	3	0	1.0	0.0 ms
Validated Improvement	3	0	1.0	0.0 ms

A limitation is that the current fact routing recognizes only the selected fixture-code and related-note question patterns. It safely declines unsupported questions, but a broader domain would need more reference facts and more general question-to-fact matching.

Generative AI Use Disclosure

1. What I used an AI assistant for and what I did myself

I used an AI assistant to help interpret the homework requirements, inspect the existing project and help debug the code. I personally ran the Redis, MySQL, and application commands, ran the semantic-cache demo and threshold sweep, reviewed the outputs, updated the related-item record, ran the multi-agent evaluation, and reviewed the saved raw files and run log.I also used the assistant to improve the text grammar and clarity of the report.

2. One AI-produced output that was wrong/unsuitable, or one thing I independently verified

I independently verified the semantic-cache threshold results rather than relying only on an AI summary. In particular, I checked the false hits where similar questions about fixture 1 and fixture 3 had different correct answers.

3. How I detected the problem or verified the result

I compared the fixed labels in `cache_cases.yaml` with the per-case results in `threshold_sweep.json` and checked the total-cost calculation separately. I also reviewed the multi-agent output to confirm that validation improved from 1 of 4 correct answers to 4 of 4 before the lessons were kept.

4. What I changed and why it works now

Ananya Yallapragada
019142383

I used the measured threshold results in the metrics table and retained the original case labels. also kept the reusable lessons only after the validation results showed improvement without changing the held-out test data.